

Universidad de San Carlos de Guatemala
Organización de lenguajes y compiladores 2
Sección "N"

Manual técnico

Nombres y Apellidos	Carné
José David Góngora Olmedo	202201444

Índice

Descripción.....	3
Lenguaje	3
Herramientas	3
PeggyJS	3
Tailwind	4
Proyecto.....	4
Vista.....	4
Index.html	4
Index.js.....	5
Analizador	6
Nodos.....	7
Gramática	8
Visitor.....	9
Entorno	9
Interprete	10
Sentencias de transferencia	10
Invocable	11
Funcion Foranea.....	12
Simbolo.....	13
Utilidades.....	13
Compilador.....	14
Generador.....	15
Registros	15
Builtins.....	16

Descripción

Proyecto que consiste en crear un lenguaje de programación, un intérprete sencillo, con las funcionalidades principales para que sea funcional.

Lenguaje

- Javascript

Herramientas

Se utilizaron librerías para el desarrollo del sistema para un mejor funcionamiento.

PeggyJS

Peggy es un sencillo generador de analizadores sintácticos para JavaScript que produce analizadores rápidos con excelentes informes de errores. Puede utilizarlo para procesar datos complejos o lenguajes informáticos y construir transformadores, intérpretes, compiladores y otras herramientas fácilmente.

Instalación

Installation

Note: When you pre-generate a grammar using the [Peggy Command Line Interface](#), no runtime is required, and Peggy can be a development-only dependency.

Node.js

To use the `peggy` command:

```
$ npm install --save-dev peggy
```

```
$ npx peggy --help
```

In your `package.json` file, you can do something like:

```
{
  "scripts": {
    "parser": "peggy -o lib/parser.js --format es src/parser.peggy"
  }
}
```

Por las necesidades del proyecto, solo se utilizó el comando **npx peggy**, para la generación del parser.

Tailwind

Tailwind es un framework CSS que nos proporciona clases de utilidad de un solo propósito que son opinables en su mayor parte, y que nos ayudan a diseñar nuestras páginas web desde dentro de nuestro marcado o archivos .js/.jsx/.ts/.tsx.

Se utilizó el CDN para aplicar los estilos en el proyecto.

<https://cdn.tailwindcss.com>

Proyecto

Este proyecto está desarrollado en Javascript Vanilla, sin el uso de ningún framework. Este fue desplegado en Github Pages y se utilizó la herramienta PeggyJS para generar el parser. El proyecto se podría dividir en dos partes, la vista y el analizador.

Vista

En esta parte del proyecto se encuentra el index.html y el index.js. En estos dos archivos se encuentra todo lo necesario para poder ver la página principal del proyecto y que todo tenga funcionalidad.

Index.html

En el archivo html se encuentra toda la estructura html para la página principal.

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <script src="https://cdn.tailwindcss.com"></script>
7   <title>Oakland</title>
8
9   <script>
10     tailwind.config = {
11       theme: {
12         extend: {
13           colors: {
14             gris: '#343a40',
15             grist: '#4C4F53',
16           }
17         }
18       }
19     }
20   </script>
21 </head>
22 <body class="h-screen flex flex-col overflow-hidden">
23   <!-- HEADER -->
24   <header class="flex-shrink-0">
25     <!-- OPCIONES MENU -->
26     <div class="w-full bg-gris">
27       <section class="flex flex-row justify-around divide-x divide-gray-500 h-full align-middle text-white content-center items-center">
28         <p id="p-archivos" class="flex flex-col cursor-pointer text-lg text-center w-full h-full hover:bg-grist align-middle justify-center">Archivo</p>
29         <p id="p-reportes" class="flex flex-col cursor-pointer text-lg text-center w-full h-full hover:bg-grist align-middle justify-center">Reportes</p>
30         <p id="p-ejecutar" class="flex flex-col cursor-pointer text-lg text-center w-full h-full hover:bg-grist align-middle justify-center">Ejecutar</p>
31       </section>
32
33       <div id="dropdown-archivos" class="bg-grist absolute z-50 w-1/3 flex flex-col text-white hidden">
34         <button id="btn-nuevo" class="hover:bg-gris h-8">Nuevo archivo</button>
35         <button id="btn-abrir" class="hover:bg-gris h-8">Abrir archivo</button>
36         <input id="fileInput" type="file" class="hover:bg-gris h-8 mx-auto hidden" multiple={false}/>
37         <button id="btn-guardar" class="hover:bg-gris h-8">Guardar archivo</button>
38       </div>

```

Index.js

En este archivo se encuentra todo el código para darle funcionalidad a la página. También es donde se encuentran todos los elementos para poder generar y mostrar los reportes.



```
1  const sidebar = document.getElementById('sidebar')
2
3  let fileCounter = 0
4  const filesList = []
5  let currentFile = null
6  export const errores = []
7  export const simbolos = []
8
9  const editor = monaco.editor.create(
10     document.getElementById('editor'), {
11         value:'',
12         language: 'java',
13         theme: 'vs-dark',
14         automaticLayout: true
15     }
16 )
17
18 const consola = monaco.editor.create(
19     document.getElementById('consola'), {
20         value:'',
21         language: 'java',
22         theme: 'vs-dark',
23         readOnly: true,
24         automaticLayout: true
25     }
26 )
```

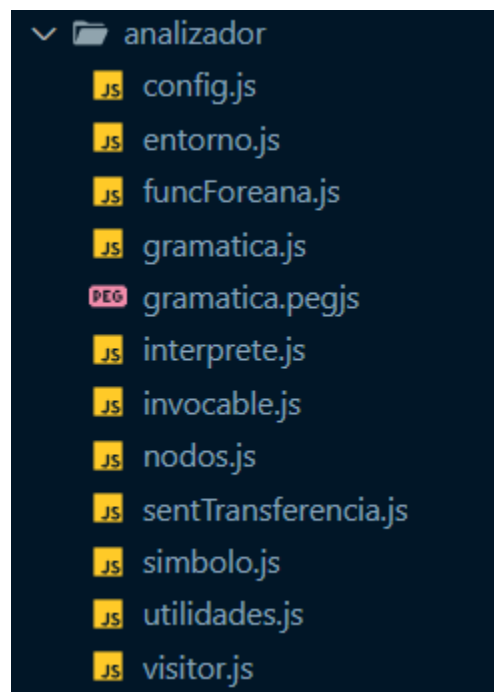
Analizador

En esta parte del proyecto se encuentra toda la lógica para interpretar la entrada.

Aquí se pueden observar todos los archivos que fueron necesarios para el funcionamiento correcto del interprete.

Para el analizador se utilizó el patrón de diseño Visitor para un manejo más fácil y mejor funcionamiento.

Visitor es un patrón de diseño de comportamiento que te permite separar algoritmos de los objetos sobre los que operan.



Nodos

En este archivo están todos los nodos que se utilizan en el intérprete, algunas producciones en la gramática retornan un nodo, el cual es utilizado para manejar todo el intérprete.

```

1  export class OperacionBinaria extends Expresion {
2
3      /**
4       * @param {Object} options
5       * @param {Expresion} options.izq Expresion izquierda de la operacion
6       * @param {Expresion} options.der Expresion derecha de la operacion
7       * @param {string} options.op Operador de la operacion
8       */
9      constructor({ izq, der, op }) {
10         super();
11
12         /**
13          * Expresion izquierda de la operacion
14          * @type {Expresion}
15          */
16         this.izq = izq;
17
18         /**
19          * Expresion derecha de la operacion
20          * @type {Expresion}
21          */
22         this.der = der;
23
24         /**
25          * Operador de la operacion
26          * @type {string}
27          */
28         this.op = op;
29
30     }
31
32     /**
33      * @param {BaseVisitor} visitor
34      */
35     accept(visitor) {
36         return visitor.visitOperacionBinaria(this);
37     }
38 }
39
40 
```

Gramática

En este archivo se encuentra toda la gramática hecha en PeggyJS y donde se retornan los nodos dependiendo de la producción que sea.

```

1  start = _ global: Global* _ { return global }
2
3  Global =
4    struct: Struct { return struct }
5    / dcl: Declaracion { return dcl }
6
7  Declaracion =
8    declArray: DeclArray _ { return declArray }
9    / decl: VarDcl _ { return decl }
10   / inst: Instancias { return inst }
11   / dclFunc: DclFunc _ { return dclFunc }
12   / sen: Sentencias _ { return sen }
13
14  VarDcl =
15    "var" _ id: ID _ "=" _ exp: Expresion _ ";" { return crearNodo('declaracionVar', { id, exp }) }
16    / tipo: TipoDato _ id: ID _ exp: ("=" _ exp: Expresion { return exp })? _ ";" { return crearNodo('declaracionVarTipo', { tipo, id, exp }) }
17

```

Al tener la gramática se ejecutaba el comando mencionado anteriormente para generar el parser con la herramienta PeggyJS.

```

1  import nodos from "./nodos.js";
2
3
4  function peg$subclass(child, parent) {
5    function C() { this.constructor = child; }
6    C.prototype = parent.prototype;
7    child.prototype = new C();
8  }
9
10 function peg$SyntaxError(message, expected, found, location) {
11   var self = Error.call(this, message);
12   // istanbul ignore next Check is a necessary evil to support older environments
13   if (Object.setPrototypeOf) {
14     Object.setPrototypeOf(self, peg$SyntaxError.prototype);
15   }
16   self.expected = expected;
17   self.found = found;
18   self.location = location;
19   self.name = "SyntaxError";
20   return self;
21 }

```


Visitor

En este archivo se encuentran todos los métodos visitor que se hayan declarado.

```

1  export class BaseVisitor {
2
3
4      /**
5       * @param {Expresion} node
6       * @returns {any}
7       */
8      visitExpresion(node) {
9          throw new Error('Metodo visitExpresion no implementado');
10     }
11
12
13     /**
14      * @param {OperacionBinaria} node
15      * @returns {any}
16      */
17     visitOperacionBinaria(node) {
18         throw new Error('Metodo visitOperacionBinaria no implementado');
19     }

```

Entorno

En el entorno como su nombre lo indica, se manejan los entornos y declaración, la edición y obtención de variables. Cada entorno tiene su entornoPadre, un nombre, sus valores, y estructuras.

```

1  export class Entorno {
2      /**
3       *
4       * @param {Entorno} entornoPadre
5       */
6      constructor(entornoPadre = undefined, nombre = "") {
7          this.valores = {}
8          this.estructuras = {}
9          this.entornoPadre = entornoPadre
10         this.nombre = nombre
11     }
12
13     /**
14      * @param {string} nombre
15      * @param {any} valor
16      */
17     setVariable(tipo, nombre, valor, location) {
18         if(this.valores[nombre]) throw new ErrorSemantico(`Variable ${nombre} ya esta declarada.`, location)
19
20         this.valores[nombre] = {valor, tipo}
21         if(valor instanceof FuncionForeanea){
22             simbolos.push(new Simbolo(nombre, "Funcion", tipo, this.nombre, location.start.line, location.start.column))
23         } else {
24             if(!((simbolos.find(item => item.id === nombre && item.ambito === this.nombre))){
25                 simbolos.push(new Simbolo(nombre, "Variable", tipo, this.nombre, location.start.line, location.start.column))
26             }
27         }
28         // simbolos.push(new Simbolo(nombre, , EncontrarTipo(valor), "variable", location.start.line, location.start.column))
29     }

```

Interprete

Este archivo va a extender del archivo Visitor, y así poder tener todos los métodos declarados en el visitor. El intérprete maneja todos los métodos visitor declarados y así darles funcionalidad.

```

1  /**
2   * @type { BaseVisitor['visitDeclaracionVar']}
3   */
4   visitDeclaracionVar(node) {
5     const nombre = node.id
6     const valor = node.exp.accept(this)
7
8     // const tipo = EncontrarTipo(valor)
9     if(!IntepreteVisitor.palabrasReservadas.includes(nombre)) throw new ErrorSemantico('Error, no se puede declarar una variable con el nombre de una palabra reservada($nombre)', node.location)
10
11     if(valor.valor === null) throw new ErrorSemantico('Error, no se puede asignar un valor nulo a la variable $nombre', node.location)
12     this.entornoActual.setVariable(valor.tipo, nombre, valor.valor, node.location)
13   }

```

Sentencias de transferencia

Aquí se encuentran las sentencias como Break, Continue y Return que extienden de Error para que así se pueda capturar por medio de un try catch y devolverlo para manejar cada sentencia. Aparte se agregó ErrorSemantico para así poder recuperar los errores semánticos encontrados al interpretar el código de entrada.

```

1  export class ExcepcionBreak extends Error {
2    constructor() {
3      super('Break');
4    }
5  }
6
7  export class ExcepcionContinue extends Error {
8    constructor() {
9      super('Continue');
10   }
11 }
12
13 export class ExcepcionReturn extends Error {
14   /**
15    * @param {any} value
16    */
17   constructor(value) {
18     super('Return');
19     this.value = value;
20   }
21 }
22
23 export class ErrorSemantico extends Error {
24   /**
25    * @param {string} message
26    */
27   constructor(message, location) {
28     super(message);
29     this.location = location
30     this.tipo = "Semantico"
31   }
32 }

```

Invocable

En este archivo se encuentra la clase de la cual van a extender las funciones. Esta clase contiene todos los métodos que deben de tener las funciones.

A screenshot of a code editor with a dark blue background and light blue text. The code is for a class named 'Invocable'. It has two methods: 'aridad()' and 'invocar()'. Both methods throw a new 'Error' with the message 'Método no implementado'. The code is numbered from 1 to 16 on the left side. The editor has three colored window control buttons (red, yellow, green) in the top left corner.

```
1  export class Invocable {
2
3
4      aridad() {
5          throw new Error("Método no implementado");
6      }
7
8      /**
9       *
10      * @param interprete {IntepreteVisitor}
11      * @param args {any[]}
12      */
13     invocar(interprete, args) {
14         throw new Error("Método no implementado");
15     }
16 }
```

Funcion Foranea

En este archivo se encuentra la clase que extiende de Invocable y así poder manejar toda la lógica de una función.

La función de aridad retorna los parámetros de la función, para así poder hacer todas las validaciones necesarias para los parámetros.

La función invocar maneja toda la lógica de la función, como su entorno y el try catch para recuperar alguna sentencia de transferencia.

```
1  export class FuncionForeanea extends Invocable {
2      constructor(node, clousure) {
3          super();
4
5          /**
6           * @type {DclFunc}
7           */
8          this.node = node;
9
10         /**
11          * @type {Entorno}
12          */
13         this.clousure = clousure;
14     }
15
16     aridad() {
17         return this.node.params
18     }
```

Simbolo

Este archivo se utilizó para crear una clase para los símbolos, el cual se utiliza al declarar alguna variable o función y así mostrarla en la tabla de símbolos.

```
1  export class Simbolo {
2      constructor(id, tipoSimbolo, tipoDato, ambito, linea, columna) {
3          this.id = id
4          this.tipoSimbolo = tipoSimbolo
5          this.tipoDato = tipoDato
6          this.ambito = ambito
7          this.linea = linea
8          this.columna = columna
9      }
10 }
```

Utilidades

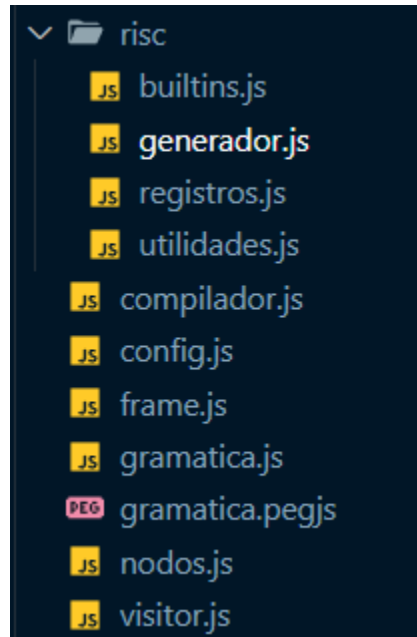
En este archivo se crearon funciones utilizadas en varios métodos visitor.

```
1  export function ValorPorDefecto(tipo) {
2      switch (tipo) {
3          case "int":
4              return 0
5          case "float":
6              return 0.0
7          case "string":
8              return ""
9          case "boolean":
10             return true
11          case "char":
12             return ''
13          default:
14             throw new Error(`Tipo ${tipo} no es valido`)
15      }
16 }
```

Compilador

En esta parte del proyecto se encuentra toda la lógica para compilar la entrada.

Se pueden ver todos los archivos necesarios para compilar de alto nivel a RISC-V. Para un manejo fácil y mejor funcionamiento también se utilizó el patrón visitor.



Generador

En este archivo se agrego todo lo necesario para agregar instrucciones a un arreglo para así devolver las instrucciones en riscv.

```

1  export class Generador {
2    constructor() {
3      this.instrucciones = []
4      this.stackObject = []
5      this.depth = 0
6      this.contLabel = 0
7      this._usedBuiltins = new Set()
8      this.arrayCount = []
9      this.funcInstrucciones = []
10   }
11
12   add(rd, rs1, rs2) {
13     this.instrucciones.push(new Instruccion("add", rd, rs1, rs2))
14   }
15
16   sub(rd, rs1, rs2) {
17     this.instrucciones.push(new Instruccion("sub", rd, rs1, rs2))
18   }

```

Registros

En este archivo se encuentran todos los registros disponibles que podemos utilizar en riscv.

```

1  export const registers = {
2    ZERO: 'zero',
3    RA: 'ra',
4    SP: 'sp',
5    S0: 's0',
6    GP: 'gp',
7    TP: 'tp',
8    T0: 't0',
9    T1: 't1',
10   T2: 't2',

```

Builtins

En este archivo se encuentran todos los builtins que vamos a necesitar a la hora de hacer la traducción de alguna instrucción.

```
1  /**
2   *
3   * @param {Generador} codigo
4   */
5  export const concatenacionString = (codigo) => {
6    codigo.push(reg.HP)
7
8    const loop1 = codigo.addLabel()
9    const end1 = codigo.getLabel()
10
11    codigo.lb(reg.T1, reg.A0)
12    codigo.beq(reg.T1, reg.ZERO, end1)
13
14    codigo.sb(reg.T1, reg.HP)
15    codigo.addi(reg.HP, reg.HP, 1)
16    codigo.addi(reg.A0, reg.A0, 1)
17    codigo.j(loop1)
18    codigo.addLabel(end1)
19
20    const loop2 = codigo.addLabel()
21    const end2 = codigo.getLabel()
22
23    codigo.lb(reg.T1, reg.A1)
24    codigo.beq(reg.T1, reg.ZERO, end2)
25    codigo.sb(reg.T1, reg.HP)
26    codigo.addi(reg.HP, reg.HP, 1)
27    codigo.addi(reg.A1, reg.A1, 1)
28    codigo.j(loop2)
29    codigo.addLabel(end2)
30
31    codigo.sb(reg.ZERO, reg.HP)
32    codigo.addi(reg.HP, reg.HP, 1)
33  }
```